

피지컬AI 과제보고서



과목	피지컬 AI
지도교수	최용훈 교수님
학번	2021741073
이름	김의성

1. 서론

기존 RacoonBot OpenVLA 환경은 색상이 다른 원통형(cylinder) 객체를 잡는(grasp) 작업만 수행할 수 있도록 구성되어 있다. 또한 언어 명령은 단일 템플릿만 사용하며, 수행 가능한 작업 역시 grasp에 한정되어 있어 데이터 다양성이 부족하다는 한계가 있다.

본 과제에서는 기존 환경의 한계를 개선하기 위해 데이터셋 확장과 추론 파이프라인 개선을 수행하였다. 데이터셋 측면에서는 새로운 객체인 sphere를 추가하고, grasp 이후 객체를 들어 올리는 lift 작업을 구현하였다. 또한 다양한 언어 표현을 사용할 수 있도록 명령어 템플릿을 확장하였다.

추론 파이프라인 측면에서는 JPEG 압축을 적용하여 이미지 전송량을 감소시키고, settle time을 단축하여 실행 시간을 줄였다. 또한 Timing Logger를 구현하여 추론 시간을 기록하고 분석할 수 있도록 하였다.

이를 통해 기존 OpenVLA 환경보다 다양한 객체와 작업을 다룰 수 있는 학습 환경을 구축하고 추론 과정의 효율성과 분석 가능성을 향상시키고자 하였다.

2. Dataset Extension

2.1 sphere 객체 추가

기존 환경에서는 원통형 객체만 존재하였기 때문에 모델이 특정 형태의 객체에만 학습되는 한계가 있었다. 이를 개선하기 위해 MuJoCo 환경에서 sphere 객체를 새롭게 추가하였다. 추가된 sphere 객체는 기존의 cylinder와 동일하게 red, blue, green, yellow의 4가지 색상으로 구성하였다. Racoon_extended_scene.XML 파일을 수정하여 각 sphere 객체를 독립적으로 움직일 수 있는 freejoint 형태로 생성하였으며, cylinder와 동일한 질량 및 충돌 특성을 적용하여 일관된 물리 환경을 유지하였다. 또한 데이터 생성 과정에서 cylinder와 sphere가 함께 배치될 수 있도록 데이터 수집 스크립트를 수정하였다. 이를 통해 기존의 4종의 cylinder 객체에서 총 8종(cylinder 4종 + sphere 4종)의 객체를 사용할 수 있도록 환경을 확장하였다.

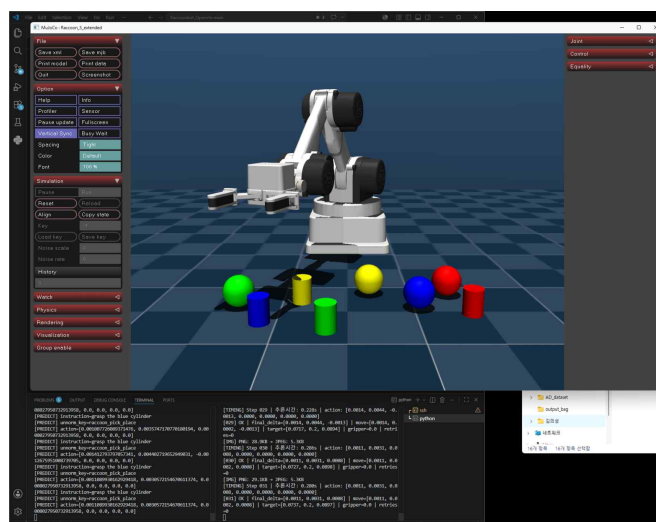


그림 1 sphere 객체 추가된 MuJoCo 환경

2.2 Lift 작업 추가

기존 데이터셋은 객체를 잡는 동작만 포함하고 있었다. 그러나 실제 조작 작업에서는 객체를 잡은 이후 이동시키거나 들어올리는 과정이 필요하다. 이를 위해 grasp 이후 객체를 일정 높이까지 들어 올리는 lift 작업을 새롭게 정의하였다. Lift 작업은 먼저 목표 객체 상단으로 이동한 후 객체 위치까지 하강한다. 그 이후 그리퍼를 닫아 객체를 파지하고 약 12cm 높이까지 상승하여 작업을 마무리한다.

이를 위해 이를 위해 새로운 데이터 수집 스크립트(raccoon_lift_dataset.py)를 구현하여 lift trajectory를 생성하였다. 또한 객체의 높이가 일정 수준 이상 상승하고 그리퍼가 닫힌 상태를 성공 조건으로 정의하여 데이터 품질을 검증하였다.

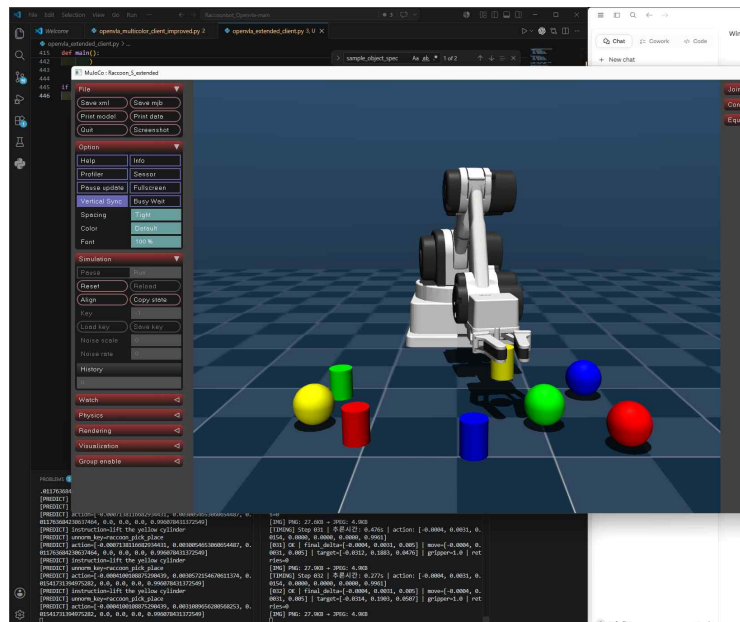


그림 2 Lift 작업 수행 예시

2.3 언어 명령 다양화

기존 데이터셋은 하나의 작업에 대한 단일 언어 템플릿만 사용하였다. 이러한 방식은 언어 표현의 다양성이 부족하여 모델이 특정 문장 패턴에 과적합될 수 있다는 한계가 있다. 본 과제에서는 동일한 작업을 여러 방식으로 표현할 수 있도록 언어 명령 템플릿을 확장하였다. 예를 들어 cylinder grasp 작업의 경우 "grasp the red cylinder", "pick up the red cylinder", "grab the red cylinder", "take the red cylinder", "get the red cylinder"로 확장하였고 sphere grasp 작업에는 "grasp the blue sphere", "pick up the blue sphere", "grab the blue sphere", "take the blue sphere" 등의 표현을 사용하였으며, lift 작업에는 "lift the green cylinder", "raise the green cylinder", "pick up and lift the green cylinder", "elevate the green cylinder" 등의 표현을 사용하였다. 이를 통해 데이터셋 내 언어 다양성을 증가시키고, 다양한 사용자 명령에 대한 일반화 성능 향상을 기대할 수 있다.

2.4 데이터셋 생성

데이터 생성 과정에서는 특정 객체나 색상에 데이터가 편향되지 않도록 cylinder 4종과

sphere 4종을 포함한 총 8개의 객체-색상 조합에 대해 균등한 비율로 에피소드를 생성하였다. 또한 객체 간 최소 거리 조건을 적용하여 객체들이 서로 겹치거나 충돌하는 상황을 방지하였으며, 각 에피소드마다 객체 위치와 자세를 무작위로 배치하여 다양한 환경을 구성하였다. 생성된 데이터는 이미지, 로봇 관절 상태, 그리퍼 상태, 객체 위치, 행동(action) 정보를 포함하며, 이후 RLDS 형식으로 변환한 뒤 TFDS 빌드를 수행하여 OpenVLA 학습에 사용할 수 있는 형태로 구성하였다. 최종적으로 grasp 작업 데이터 300 에피소드와 lift 작업 데이터 150 에피소드를 수집하였으며, 이를 통합하여 총 450 에피소드 규모의 raccoon_mixed 데이터셋을 구축하였다.

생성된 에피소드를 검증하기 위해 PNG 프레임 시퀀스를 GIF로 변환하는 시각화 도구를 구현하였다. 이를 통해 생성된 trajectory와 instruction을 직관적으로 확인할 수 있었으며, 데이터 생성 과정의 정상 동작 여부를 검증하였다.

확장된 데이터셋은 기존 환경보다 더 다양한 객체, 작업, 언어 명령을 포함하며 OpenVLA 모델의 학습 데이터 다양성을 향상시킬 수 있다.

3. Code Improvement

데이터셋 확장과 함께 OpenVLA 추론 파이프라인의 효율성을 향상시키기 위한 코드 개선을 수행하였다. 기존 클라이언트는 cylinder grasp 작업만 지원하였으며, 이미지 전송 효율과 추론 과정 분석 기능이 부족하다는 한계가 있었다. 본 과제에서는 새로운 객체 및 작업 지원과 함께 추론 성능 개선을 위한 기능을 추가하였다.

3.1 JPEG 기반 이미지 압축

기존 클라이언트는 카메라 영상을 PNG 형식으로 서버에 전송하였다. PNG는 무손실 압축 방식이지만 파일 크기가 상대적으로 크기 때문에 네트워크 전송 시간이 증가할 수 있다. 이를 개선하기 위해 이미지 전송 방식을 JPEG 압축 방식으로 변경하였다. JPEG 품질 계수를 50으로 설정하여 시각적 품질은 유지하면서 데이터 크기를 효과적으로 감소시켰다.

실험 결과 이미지 크기는 평균적으로 약 27KB에서 5KB 수준으로 감소하였으며, 약 80% 이상의 전송량 감소 효과를 확인할 수 있었다. 이를 통해 서버와 클라이언트 간 데이터 전송 효율을 향상시킬 수 있었다.

3.2 Settle Time 최적화

기존 파이프라인에서는 각 행동(action) 수행 이후 환경이 안정화될 때까지 0.8초의 대기 시간을 사용하였다. 그러나 실제 실험 결과 0.8초의 대기 시간은 필요 이상으로 길어 전체 실행 시간을 증가시키는 요인으로 작용하였다. 따라서 환경이 충분히 안정화되는 범위 내에서 settle time을 0.2초로 감소시켰다. 이를 통해 에피소드당 불필요한 대기 시간을 줄일 수 있었으며, 전체 추론 및 실행 속도를 향상시킬 수 있었다.

3.3 Timing Logger 구현

기존 클라이언트는 추론 시간이 얼마나 소요되는지 확인할 수 있는 기능이 존재하지 않았다. 따라서 성능 분석이나 병목 구간 파악이 어려웠다. 이를 해결하기 위해 Timing Logger를 구현하여 각 추론 단계의 실행 시간을 기록하도록 하였다. 측정된 추론 시간은 CSV 파

일 형태로 저장되며, 에피소드 종료 시 평균 추론 시간, 최소 추론 시간, 최대 추론 시간을 자동으로 계산하여 출력하도록 구성하였다. 모델의 추론 성능을 정량적으로 분석할 수 있으며, 향후 추가적인 최적화 작업을 위한 기준 지표로 활용할 수 있다.

3.4 추론 결과 시각화

추론 과정의 동작을 쉽게 확인하기 위해 rollout 결과를 GIF 형태로 변환하는 시각화 기능을 추가하였다. 각 에피소드에서 저장된 프레임 이미지를 이용하여 GIF를 생성하도록 구현하였으며, 이를 통해 OpenVLA가 생성한 행동 시퀀스를 직관적으로 확인할 수 있었다. 또한 모델의 실패 사례나 비정상 동작을 분석하는 데에도 활용할 수 있다.

4. Results and Discussion

그림 1은 sphere 객체가 추가된 MuJoCo 환경을 보여준다. 기존 cylinder 객체 외에 4개의 sphere 객체를 추가하여 총 8개의 객체를 배치하였으며, 이를 통해 보다 다양한 형태의 객체에 대한 학습 데이터를 생성할 수 있었다.

그림 2는 생성된 데이터셋을 이용하여 sphere 객체를 파지하는 장면을 보여준다. 기존 cylinder 객체뿐만 아니라 sphere 객체에 대해서도 정상적으로 grasp 동작을 수행할 수 있음을 확인하였다.

그림 3은 OpenVLA 추론 결과로 생성된 lift 작업 수행 장면이다. 모델은 주어진 자연어 명령에 따라 목표 객체를 선택하고 파지한 뒤 들어 올리는 동작을 수행하였다. 또한 JPEG 압축 및 Timing Logger를 포함한 개선된 추론 파이프라인이 정상적으로 동작함을 확인할 수 있었다.

본 과제에서는 추가적으로 pick-and-place 작업 구현도 시도하였다. 그러나 grasp 이후 목표 위치까지 이동하는 과정에서 성공률이 매우 낮게 나타났으며, 안정적인 데이터 수집이 어려워 최종 데이터셋에는 포함하지 못하였다.

또한 기존 4개 객체 환경과 달리 총 8개의 객체를 동시에 배치하면서 객체 간 간격이 좁아지는 문제가 발생하였다. 이로 인해 그리퍼가 주변 객체와 충돌하거나 접근 경로가 방해되는 경우가 자주 발생하였다. 특히 작업 공간 범위가 넓어지면서 일부 객체는 로봇이 안정적으로 접근하기 어려운 위치에 배치되었고, 결과적으로 grasp 성공률이 감소하는 문제가 발생하였다.

이 문제를 해결하기 위해 그리퍼 자세와 접근 각도를 함께 제어하는 방안을 고려하였다. 하지만 제한된 시간 내에 안정적으로 구현하지 못하여 최종 데이터셋에는 반영하지 못하였다. 향후에는 end-effector의 orientation까지 고려한 trajectory 생성 기법을 적용하여 보다 안정적인 데이터 수집을 수행할 계획이다.

또한 OpenVLA 모델의 fine-tuning 과정에도 충분한 시간을 투자하지 못한 점이 아쉬웠다. 다양한 하이퍼파라미터 실험과 장시간 학습을 수행한다면 성능을 더욱 향상시킬 수 있었을 것으로 예상된다. 다만 과제 기간 내에는 데이터셋 구축과 파이프라인 확장에 집중하여 기본적인 학습 및 추론 검증을 수행하였다.

과제를 통해 Physical AI 파이프라인의 전체 과정을 경험할 수 있었다. 단순히 모델을 사용하는 수준을 넘어 MuJoCo 환경 수정, 데이터 생성, RLDS 변환, OpenVLA fine-tuning, 추론 및 평가 과정을 직접 수행하면서 VLA 모델이 실제로 학습되는 과정을 이해할 수 있었다. 특

히 데이터 품질이 모델 성능에 미치는 영향과 데이터 생성 과정의 중요성을 체감할 수 있었으며, Physical AI 연구에 대한 이해를 넓힐 수 있는 의미 있는 경험이었다.